Agentic User Modelling and Conversational Recommendation: A Multi-Step LLM Pipeline for the DSN × BCT Challenge

Adeniyi Oluwademiladeayo Samuel, Oyebamire Oluwaseun, Ukegbu Chidera

Abstract. We present a unified, two-task agentic system addressing the DSN × BCT LLM Agent Challenge. Task A simulates a user's star rating and written review for unseen items by combining a behavioural-profile analysis step with in-voice generation. Task B delivers personalised recommendations via a retrieve-then-rerank pipeline over a 5,000-item Amazon Beauty catalog, supporting cold-start and multi-turn refinement. Both agents run inside a single FastAPI service exposing two HTTP endpoints, packaged as a reproducible Docker container. We evaluate on the McAuley Lab Amazon Reviews 2023 dataset using a leave-last-out split. The system also supports a Nigerian-context persona overlay for the bonus track.

1. Problem framing

Online review platforms encode rich, latent user behaviour: rating tendencies, preferred topics, stylistic patterns, and shifting context. Two complementary challenges arise from this data: **user modelling** — given a user's review history, can a system simulate the rating and review they would give to a new item, preserving their voice and harshness? — and **recommendation** — given a user's preferences, can a system rank items likely to delight them, even in cold-start or contextual settings?

Both tasks resist conventional collaborative filtering: ratings alone strip away tone, and pure embedding similarity ignores reasoning about why a user might prefer one item over another. Our approach treats each user as a dynamic agent, using a large language model to perform explicit chain-of-reasoning steps before producing outputs.

2. Task A — Review Simulator Agent

2.1 Architecture

`ReviewSimulatorAgent` decomposes the task into two LLM calls plus a structured-output step:

1. Persona analysis. The agent first describes the user's reviewing personality in 2–3 sentences (lenient vs strict; brief vs verbose; what they tend to care about). This makes implicit signals — e.g. *'this user consistently rates 5 stars but mentions ingredient quality'* — explicit before the generation step.

2. Behaviour-grounded generation. The persona text, the personality analysis, and the new target item metadata are concatenated into a single prompt that asks for (a) an integer rating in [1, 5], and (b) the review text the user would have written, at their typical length. Output is constrained to a JSON object that includes `predicted_rating`, `review_text`, and a one-sentence `reasoning` rationale.

This two-step factoring is the key design choice. We found that asking the LLM to 'be the user' without an explicit personality step produces generic, tone-neutral reviews. The analysis step acts as a chain-of-thought prefix specifically targeted at the user model.

Persona pack. Each user's history is summarised into a compact JSON profile: their average rating, rating distribution, average review length, and up to 10 of their most recent reviews (title, rating, 200-character excerpt, date). This keeps prompts under ~1,500 tokens per call.

Metric	Value
RMSE (rating)	1.18
MAE (rating)	0.93
Exact-match rating	38%
Within ±1 star	78%
ROUGE-L (review text)	0.142

2.2 Results (n = 15 held-out users, All_Beauty)

The agent achieves 78% rating agreement within one star, with most error concentrated on users whose rating distribution skews toward the extremes. ROUGE-L is moderate because user reviews are short and stylistically varied; high overlap would require memorising training reviews verbatim, which we explicitly avoid.

2.3 Nigerian-context overlay (bonus track)

The agent accepts a `naija_mode` flag that injects a persona overlay instructing the LLM to use natural Nigerian English with comfortable code-switching and light pidgin. The flag does not change the architecture; it only shapes the tone of the generated review. We tested it on three hand-crafted Nigerian personas (Chiamaka, Tunde, Zainab) reviewing three Nigeria-relevant products (Kano-made shea butter, generator-compatible clipper, sunscreen for melanin-rich skin). The generated reviews preserved each persona's harshness profile while adopting the requested register without caricature. See `results/naija_showcase.json` for the full outputs.

3. Task B — Recommendation Agent

3.1 Architecture

`RecommendationAgent` follows a three-stage retrieve-then-rerank pattern:

1. Intent inference. The LLM converts the user's history (and optional `conversation_context`) into a single specific natural-language query describing what they are likely to want next.

2. Semantic retrieval. The query is encoded with `sentence-transformers/all-MiniLM-L6-v2` and matched against the catalog via a FAISS `IndexFlatIP` (cosine similarity on L2-normalised vectors). We retrieve the top 200 items.

3. LLM reranking. The top 40 retrieval results, the user's profile, and the user's recent ratings are sent to the LLM, which selects the final top-10 with one-sentence rationales referencing the specific past purchases that motivate each pick.

The rationale step is what makes this agent recognisably different from collaborative filtering: every recommendation is accompanied by a grounded reason such as *'the user's 5-star review of the foot peel mask suggests they will appreciate this exfoliating body scrub.'*

Fallback behaviour. When the LLM's response is malformed or hallucinates asins not in the candidate set,

the agent gracefully falls back to retrieval order with a `retrieval_fallback` reason. This makes the endpoint robust under rate-limit or parsing failures.

3.2 Results (n = 15 held-out users, 5K-item All_Beauty catalog)

0.130
0.061
Hit@10 NDCG@10 Single-item next-purchase recall from a 5K-item catalog is genuinely difficult; a Hit@10 of 0.13 corresponds to roughly one in eight users seeing their actual next purchase surface in the top ten. This is in the same range as published baselines on similar Amazon-category benchmarks. The rubric also includes contextual-relevance human evaluation, where we expect our explicit-rationale output to perform stronger than retrieval-only baselines.

3.3 Cold-start and multi-turn

The agent natively handles two scenarios that pure collaborative filtering struggles with:
Cold-start. When a persona has zero history items, the intent-inference prompt switches to an explicit 'cold-start' branch that asks the LLM to default to broad, popular product categories rather than personalised picks. The retrieval pipeline is otherwise unchanged.
Multi-turn. The `/recommend` endpoint accepts an optional `conversation_context` string that is incorporated into the intent query. We demonstrate this by issuing two consecutive turns for the same user, where the second turn adds *'I want something specifically for dry hair'* — the recommendations correctly shift toward hair-care products without losing the user's overall stylistic profile. See `results/coldstart_multiturn.json`.

4. Engineering and deployment

The two agents share a single FastAPI service (`/simulate-review` and `/recommend`) with `/health` and `/` introspection endpoints. The catalog parquet and FAISS index are baked into the Docker image at build time so the container is fully self-contained; the only runtime dependency is a `GROQ_API_KEY` environment variable. The service starts in approximately 12 seconds on a free-tier HuggingFace Space, dominated by the sentence-transformer cold-start.
Choice of LLM provider. We initially built on Gemini 2.5 Flash but encountered free-tier rate limits during evaluation. We migrated to Llama 3.3 70B via Groq, which offered both substantially higher throughput and qualitatively better adherence to JSON output schemas on the rerank task. The agent abstraction made this swap a single-file change.

5. Ablations

What we examined.
Single-step vs two-step Task A. Removing the personality-analysis step degraded review quality on qualitative inspection — outputs became tonally generic and lost the lenient/strict calibration that defines individual users.
Retrieval depth. With k=50 candidates, recall was effectively zero on held-out targets; widening to k=200 with rerank of top-40 was necessary to bring Hit@10 above zero. This points to the embedder being undertrained for the specificity of consumer-product titles.
Catalog size. We capped the catalog at 5,000 items to keep the FAISS index fast and the rerank prompt focused. A larger catalog would expose richer items but worsen retrieval recall without a stronger embedder

References

[1] Hou, Y., Li, J., He, Z., Yan, A., Chen, X., McAuley, J. *Bridging Language and Items for Retrieval and Recommendation.* arXiv:2403.03952. (Amazon Reviews 2023 dataset.)
[2] Reimers, N., Gurevych, I. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* EMNLP 2019.
[3] Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., Jégou, H. *The Faiss library.* arXiv:2401.08281, 2024.
[4] Llama 3 Team, Meta AI. *The Llama 3 Herd of Models.* arXiv:2407.21783, 2024.